

Memory-Weighted Selection: Middleware for Continuity and Governed Behaviour in LLM Systems

Marcos Verrell
Independent Researcher

Abstract

Large language model systems often rely on direct base-model emission or retrieval-conditioned generation, which can leave continuity and policy enforcement dependent primarily on upstream training, transient context, or post-hoc moderation rather than an explicit selection mechanism. This paper proposes a formal middleware-level account of how persistent memory state can bias selection over candidate outputs from a base model. We propose memory-weighted selection: a re-ranking mechanism in which candidate outputs are scored against a structured memory state and a defined set of governor constraints before final emission. The reference formulation decomposes memory into recency, salience, and anchor components, and defines governed behaviour as behaviour selected under explicit penalties over candidate outputs. We describe a middleware architecture that applies this mechanism downstream of candidate generation and upstream of final response selection, without requiring modification of the base model. Worked scenarios show how memory weighting can alter selection relative to direct base-model emission, including a numerical example of governor-constrained candidate selection. The claim is narrow and engineering-focused: broader theoretical, cognitive, or physical interpretations are outside the scope of this paper.

1 Introduction

LLM-based systems increasingly rely on surrounding infrastructure to provide memory, retrieval, tool use, and policy control. In many deployments, however, the final output remains strongly shaped by direct base-model emission, transient prompt context, or retrieval-conditioned generation. This can produce weak behavioural continuity across turns, inconsistent treatment of long-term commitments, and policy enforcement that depends primarily on upstream training or post-hoc moderation when safety or stability constraints are applied outside the selection process.

This paper makes a narrow engineering claim: that a middleware layer applying memory-weighted selection over candidate outputs from a base model provides a defined site for continuity and governance enforcement, complementary to existing LLM infrastructure. Broader theoretical implications are outside the scope of this work.

In associated public project documentation, this style of layered memory-influence architecture is described as Weighted Emergence Layering (WEL): prior interaction events are retained as structured behavioural weights that can influence later candidate selection under governor control. This paper uses the more general term memory-weighted selection for the formal reference mechanism, and treats WEL as compatible public architecture terminology rather than as a disclosed implementation.

The proposed mechanism treats candidate selection as a distinct stage between base-model generation and final output emission. A base model produces a candidate set. A middleware layer scores each candidate against a persistent memory state and a defined set of governor constraints.

The selected output is the candidate that best satisfies base-model plausibility, memory alignment, and governor compliance under the reference scoring formulation.

In this paper, governed behaviour means behaviour selected under an explicit set of constraints applied at the selection step. These constraints are represented as penalties over candidate outputs, rather than as post hoc moderation alone or as token-level decoding restrictions. The term governor is used in a generic architectural sense: a policy and stability-control layer that can penalise or reject candidates inconsistent with defined constraints.

The contributions of this paper are:

1. A formal reference model for memory-weighted candidate selection, with persistent memory decomposed into recency, salience, and anchor components.
2. A middleware architecture that applies this selection downstream of candidate generation and upstream of final output emission, without requiring modification of the base model.
3. Worked scenarios showing how memory-weighted selection can alter output selection relative to direct base-model emission.

The focus is deliberately limited. The paper proposes a reference formulation and architecture, not a finished benchmark suite or production deployment report. Production-specific scoring functions, governor policies, thresholds, persistence schemas, proprietary kernels, and implementation details are outside the scope of this paper.

This paper does not claim that the proposed architecture produces consciousness, sentience, agency, or artificial general intelligence. It does not claim to validate any cognitive, physical, field-theoretic, quantum-measurement, or other scientific interpretation beyond the middleware-level selection mechanism described here. It does not claim to replace foundation models, retrieval-augmented generation, reinforcement learning from human feedback, or token-level constrained decoding.

2 Related Work

Memory and control mechanisms for LLM systems have been studied through several overlapping approaches, including memory-augmented neural networks, retrieval-augmented generation, reranking and best-of-N selection, long-horizon agent memory, constrained decoding, reinforcement learning from human feedback, and constitutional or rule-based governance methods. The framing of selection under uncertainty has antecedents in decision-making under partial observability [Kaelbling et al., 1998, Sutton and Barto, 2018] and sequential candidate selection in bandit problems [Bubeck and Cesa-Bianchi, 2012].

Memory-augmented neural networks introduced differentiable memory structures that allow models to read from and write to external memory during computation [Graves et al., 2014, 2016, Weston et al., 2015, Sukhbaatar et al., 2015]. Retrieval-augmented generation systems condition output generation on retrieved documents or prior context [Lewis et al., 2020, Guu et al., 2020, Borgeaud et al., 2022, Izacard and Grave, 2021]. Agent memory architectures extend this idea into longer-horizon systems by storing observations, reflections, plans, or task histories across multiple interactions [Park et al., 2023, Shinn et al., 2023, Packer et al., 2023, Wang et al., 2023]. Constrained decoding and controllable generation methods influence the token-generation process directly [Dathathri et al., 2019, Krause et al., 2021, Yang and Klein, 2021, Beurer-Kellner et al., 2023, Willard and Louf, 2023], while RLHF and related alignment approaches modify model behaviour through training or preference optimisation [Ouyang et al., 2022, Bai et al., 2022b,a]. Reranking and

best-of-N approaches select among multiple candidate outputs using relevance, preference, reward, or verification criteria [Nogueira and Cho, 2019, Burges, 2010, Liu, 2009, Stiennon et al., 2020, Nakano et al., 2021, Lightman et al., 2023, Cobbe et al., 2021].

Memory-weighted selection differs from these approaches in its reference placement and mechanism. It does not require the base model to be retrained, and it does not primarily condition token generation on retrieved context. Instead, it operates over fully formed candidate outputs produced by a base model. The middleware scores candidates against a structured persistent memory state and explicit governor penalties, making selection itself the primary site of continuity and governance enforcement.

The mechanism also differs from conventional rerankers and best-of-N selection methods. Traditional rerankers typically optimise against static criteria such as relevance, correctness, or preference alignment within a single interaction. Memory-weighted selection instead scores candidates against a persistent and continuously updated memory state composed of recency, salience, and anchor components, together with explicit governor penalties. The objective is not only single-turn optimisation, but continuity-aware selection across interactions.

This distinction is important. Retrieval-augmented generation asks what information should be placed into context before generation. Token-level constraints ask which tokens should be allowed during generation. Conventional rerankers ask which candidate best satisfies a fixed ranking objective. Memory-weighted selection asks which complete candidate output should be emitted after generation, given persistent memory and governor constraints.

3 Memory-Weighted Selection Principle

The central principle is that retained state can bias future candidate selection. In the present paper, this is treated as an engineering mechanism rather than a broad theoretical claim. Related public documentation describes the proposed influence of retained prior information on future selection probability as Active Information Weight (AIW); here, that influence is represented by the memory-alignment term $s_{mem}(y | M_t)$ and the coefficient λ_M .

At each step t , the system receives an input x_t . A base model produces a candidate set. The reference formulation assumes such a set is available; in practice this may be produced through temperature sampling, n-best generation, diverse beam search, or other diversity-promoting strategies. The interaction between candidate diversity and selection quality is left to future work.

A candidate set is represented as:

$$C_t = \{y_{t,1}, y_{t,2}, \dots, y_{t,n}\}. \quad (1)$$

A middleware layer then selects one candidate for final emission. The selection is influenced by three factors: base-model score or plausibility, memory alignment, and governor penalty.

The persistent memory state is represented as:

$$M_t = (R_t, S_t, A_t), \quad (2)$$

where R_t represents recency-weighted state, S_t represents salience-weighted state, and A_t represents anchor state, including persistent commitments or stability constraints. This recency-salience-anchor decomposition is the public reference structure used in the associated middleware documentation. It also echoes structures developed in cognitive memory models [Anderson, 1996, Anderson and Schooler, 1991, Anderson et al., 2004].

Symbol	Meaning
x_t	Input at step t
C_t	Candidate output set at step t
y	Candidate output
y_t^*	Selected output at step t
M_t	Persistent memory state
R_t	Recency component of memory
S_t	Saliency component of memory
A_t	Anchor component of memory
$s_{base}(y \mid x_t)$	Base-model candidate score
$s_{mem}(y \mid M_t)$	Memory-alignment score
$p_G(y)$	Governor penalty for candidate y
λ_M	Memory influence coefficient
λ_G	Governor influence coefficient
α_R	Recency component weight
α_S	Saliency component weight
α_A	Anchor component weight

Table 1: Notation used in the reference formulation.

The selected output is:

$$y_t^* = \arg \max_{y \in C_t} [s_{base}(y \mid x_t) + \lambda_M s_{mem}(y \mid M_t) - \lambda_G p_G(y)], \quad (3)$$

where $s_{base}(y \mid x_t)$ is the base-model candidate score or estimated plausibility, $s_{mem}(y \mid M_t)$ is the memory-alignment score, $p_G(y)$ is the governor penalty assigned to candidate y , λ_M controls the influence of memory, and λ_G controls the influence of governor constraints.

The memory-alignment score is decomposed as:

$$s_{mem}(y \mid M_t) = \alpha_R r(y, R_t) + \alpha_S \sigma(y, S_t) + \alpha_A a(y, A_t), \quad (4)$$

where $r(y, R_t)$ is a recency-alignment term, for example exponentially decayed similarity between candidate representations and recent selections; $\sigma(y, S_t)$ is a saliency-alignment term, for example weighted similarity against high-saliency memory items; $a(y, A_t)$ is an anchor-alignment term, for example compatibility with persistent commitments, long-term preferences, or stability constraints; and α_R , α_S , and α_A weight the relative influence of the memory components.

This formulation is not intended to prescribe a single production scoring function. It provides a reference structure that can be instantiated with different similarity metrics, decay functions, saliency estimators, and governor policies.

4 Reference Architecture

The reference architecture commits to candidate re-ranking rather than generation conditioning. A base model first generates a set of candidate outputs. The middleware then evaluates those candidates against persistent memory and governor constraints. This framing is model-agnostic: the upstream generator may be an LLM, scripted decision layer, simulation policy, or other candidate-producing system.

The pipeline is:

1. Input received.
2. Base model generates candidate set.
3. Middleware retrieves or constructs the current memory state.
4. Each candidate is scored for base plausibility, memory alignment, and governor penalty.
5. The highest-scoring candidate under the reference objective is selected.
6. The selected output is emitted.
7. The memory state is updated according to the selected output and associated metadata.

This placement preserves compatibility with existing LLM systems. It allows memory and governor logic to be introduced as a middleware layer without requiring access to the base model weights or training process.



Figure 1: Reference architecture for memory-weighted candidate re-ranking.

5 Worked Scenarios

The following examples are illustrative rather than empirical benchmark results. Their purpose is to show how the reference mechanism changes candidate selection relative to direct base-model emission.

5.1 Continuity Across Prior Commitments

Suppose a user has repeatedly indicated a preference for concise technical responses. A direct base-model emission may select a verbose explanation because it is broadly plausible for the input. Under memory-weighted selection, candidates aligned with the persistent preference receive higher memory scores, making a concise technical response more likely to be selected.

5.2 Salience-Weighted Project Continuity

Suppose an agent is assisting with a long-running software project in which prior interactions established a commitment to maintaining compatibility with an existing framework version. A generic candidate may recommend replacing major components with incompatible alternatives. A memory-weighted selection layer can penalise candidates inconsistent with persistent project constraints and favour candidates aligned with the established continuity requirements.

5.3 Governor-Constrained Selection

Suppose a user asks for guidance on a topic where prior interactions established a specific project preference. Candidate A is direct but inconsistent with prior commitments. Candidate B is moderately phrased but aligned with prior commitments and stability constraints. Candidate C is conservative but only weakly aligned.

Suppose three candidate outputs are produced for this input. Scores below are illustrative values on a normalized unit interval; practical implementations may use different scoring ranges depending on model architecture and calibration strategy.

Candidate	Base Score	Memory Score	Governor Penalty	Final Score
A	0.82	0.18	0.35	0.44
B	0.74	0.32	0.05	0.92
C	0.69	0.22	0.02	0.84

Table 2: Illustrative candidate scores under the reference objective.

Using illustrative weights:

$$\lambda_M = 0.8, \quad \lambda_G = 1.5. \quad (5)$$

Candidate B receives:

$$r(B, R_t) = 0.32, \quad \sigma(B, S_t) = 0.41, \quad a(B, A_t) = 0.15, \quad (6)$$

with component weights:

$$\alpha_R = 0.4, \quad \alpha_S = 0.4, \quad \alpha_A = 0.2. \quad (7)$$

This gives:

$$s_{mem}(B \mid M_t) = (0.4 \times 0.32) + (0.4 \times 0.41) + (0.2 \times 0.15) \approx 0.32. \quad (8)$$

The final score for each candidate is computed as:

$$s_{final}(y) = s_{base}(y \mid x_t) + \lambda_M s_{mem}(y \mid M_t) - \lambda_G p_G(y). \quad (9)$$

Candidate A receives the highest base-model score but incurs the largest governor penalty. Candidate B receives a lower base score but stronger alignment with the persistent memory state and a lower governor penalty. Under the reference selection objective, candidate B becomes the selected output.

This example is illustrative rather than empirical. Its purpose is to show how the reference formulation alters candidate ranking relative to direct base-model emission.

6 Reference Implementation Notes

A prototype middleware instantiating the proposed architecture has been developed for internal evaluation. The present paper does not depend on production deployment claims. Instead, it describes the architectural principle and reference formulation that such a system can instantiate.

In a reference implementation, the base model or upstream generator produces multiple candidate outputs. The middleware receives these candidates, retrieves the current memory state, computes memory-alignment scores, applies governor penalties, and emits the selected candidate. After emission, the middleware records an update to the memory state, including recency metadata,

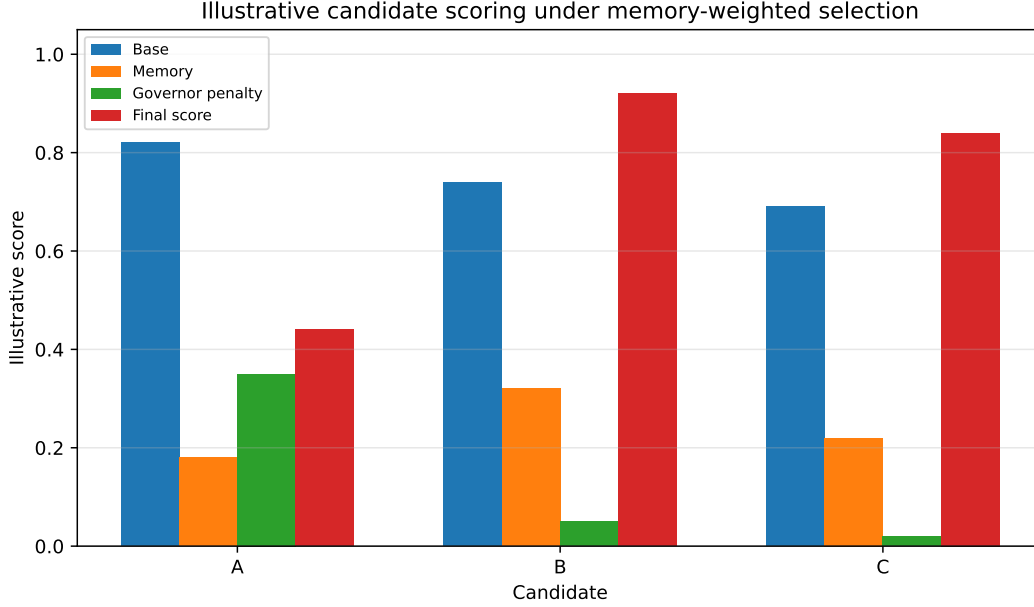


Figure 2: Illustrative comparison of base score, memory score, governor penalty, and final score.

salience indicators, and any anchor-relevant changes. In an applied software project, the same public structure may be described as governed, memory-weighted behavioural middleware: continuity memory, recency, salience, anchors, and WEL-style layered influence shape later candidate selection while private kernel logic and production thresholds remain undisclosed.

Production-specific parameterisations, internal scoring functions, governor policies, thresholds, persistence schemas, and kernel-level implementation details are outside the scope of this paper. The reference formulation is intended to be reproducible at the architectural level without disclosing proprietary implementation details.

7 Limitations

This paper presents a reference formulation and architecture, not a large-scale empirical evaluation. The worked scenarios are designed to clarify mechanism behaviour and do not constitute benchmark validation.

The formulation does not prescribe a single optimal memory-scoring function, governor policy, or parameter schedule. Different implementations may require different similarity functions, salience estimates, decay rates, and penalty functions.

The proposed architecture does not replace base models, retrieval systems, RLHF, constrained decoding, reranking systems, best-of-N selection, or moderation systems. It can be understood as a middleware-level selection mechanism that may be combined with those methods.

The paper makes no claim about consciousness, sentience, agency, cognition, physics, field-theoretic interpretation, quantum measurement, or artificial general intelligence. Its claim is limited to middleware-level candidate selection in LLM systems.

8 Future Work

Future work should focus on empirical evaluation on continuity and governance benchmarks; comparison against baseline memory architectures, retrieval-augmented systems, conventional rerankers, and direct base-model emission; characterisation of parameter sensitivity for λ_M , λ_G , and component weights α_R , α_S , α_A ; and extensions to multi-agent and longer-horizon settings.

References

- John R. Anderson. Act: A simple theory of complex cognition. *American Psychologist*, 51(4): 355–365, 1996.
- John R. Anderson and Lael J. Schooler. Reflections of the environment in memory. *Psychological Science*, 2(6):396–408, 1991.
- John R. Anderson, Daniel Bothell, Michael D. Byrne, Scott Douglass, Christian Lebiere, and Yulin Qin. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004.
- Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022a.
- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022b.
- Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2023. arXiv:2212.06094.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning*, pages 2206–2240, 2022.
- Sébastien Bubeck and Nicolo Cesa-Bianchi. *Regret Analysis of Stochastic and Nonstochastic Multi-armed Bandit Problems*. Now Publishers, 2012.
- Christopher J. C. Burges. From ranknet to lambdarank to lambdamart: An overview. *Microsoft Research Technical Report MSR-TR-2010-82*, 2010.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Sumanth Dathathri, Andrea Madotto, Janice Lan, Jane Hung, Eric Frank, Piero Molino, Jason Yosinski, and Rosanne Liu. Plug and play language models: A simple approach to controlled text generation. *arXiv preprint arXiv:1912.02164*, 2019.
- Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.

- Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwinska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538 (7626):471–476, 2016.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Realm: Retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning*, pages 3929–3938, 2020.
- Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pages 874–880, 2021.
- Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.
- Ben Krause, Akhilesh Deepak Gotmare, Bryan McCann, Nitish Shirish Keskar, Shafiq Joty, Richard Socher, and Nazneen Fatema Rajani. Gedi: Generative discriminator guided sequence generation. *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4929–4952, 2021.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- Tie-Yan Liu. *Learning to Rank for Information Retrieval*. Springer, 2009.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.
- Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*, 2019.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.
- Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 2023.

- Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize from human feedback. In *Advances in Neural Information Processing Systems*, volume 33, pages 3008–3021, 2020.
- Sainbayar Sukhbaatar, Jason Weston, Rob Fergus, et al. End-to-end memory networks. *Advances in Neural Information Processing Systems*, 28, 2015.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. *International Conference on Learning Representations*, 2015.
- Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.
- Kevin Yang and Dan Klein. Fudge: Controlled text generation with future discriminators. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 3511–3535, 2021.